

Bộ câu hỏi phản biện Apache Flink Demo

Tài liệu được sắp xếp lại theo cấu trúc **Câu hỏi** → **Trả lời** để dễ ôn và tra cứu.

Mục lục

1. Câu 1 — Vai trò của Kafka và Flink
2. Câu 2 — Kafka Partition và Source Subtask
3. Câu 3 — Source Parallelism lớn hơn số Kafka Partitions
4. Câu 4 — Partition, Subtask và Task Slot
5. Câu 5 — Parallel Processing và Distributed Processing
6. Câu 6 — HASH trong Job Graph và keyBy
7. Câu 7 — Tại sao phải keyBy trước Window
8. Câu 8 — Một triệu Key có cần một triệu Subtask?
9. Câu 9 — Hot Key và Data Skew
10. Câu 10 — Tại sao dùng Tumbling Window 10 giây?
11. Câu 11 — Tumbling Window và Sliding Window
12. Câu 12 — Window State được lưu ở đâu?
13. Câu 13 — Processing Time và Event Time
14. Câu 14 — Vai trò của Watermark
15. Câu 15 — Checkpoint thực sự lưu gì?
16. Câu 16 — Recovery khi TaskManager bị lỗi
17. Câu 17 — Kafka đã lưu dữ liệu, tại sao vẫn cần Checkpoint?
18. Câu 18 — Checkpoint có đồng nghĩa Exactly-once?
19. Câu 19 — Console Sink, Duplicate và Database Sink
20. Câu 20 — Control Flow và Data Flow
21. Câu 21 — Scale demo khi có thêm máy

Câu 1 — Vai trò của Kafka và Flink

? Câu hỏi

Em vừa nói đây là real-time transaction analytics. Vậy Kafka và Flink khác nhau ở vai trò nào? Tại sao không bỏ Kafka và cho producer gửi transaction trực tiếp vào Flink?

✓ Trả lời

Kafka và Flink khác nhau ở vai trò:

- Kafka là message broker / distributed log, dùng để nhận, lưu tạm, phân vùng và phát lại transaction.
- Flink là stream processing engine, dùng để đọc stream đó và xử lý logic như parse, filter, keyBy, window, aggregate, detect bất thường.

Không nên bỏ Kafka để producer gửi trực tiếp vào Flink vì Kafka giúp tách producer khỏi Flink. Nếu Flink restart, deploy lại hoặc bị chậm, Kafka vẫn giữ dữ liệu theo retention để Flink đọc lại từ offset đã checkpoint. Kafka cũng giúp buffer tải, chia partition để scale, và cho nhiều consumer khác cùng đọc dữ liệu.

Câu 2 — Kafka Partition và Source Subtask

? Câu hỏi

Trong demo em tạo Kafka topic có 2 partitions và Flink chạy parallelism = 2.

Vậy có phải:

Partition 0 → Source Subtask 0

Partition 1 → Source Subtask 1

là mapping cố định không? Nếu thầy đổi thành 10 Kafka partitions nhưng Source parallelism = 2 thì chuyện gì xảy ra?

✓ Trả lời

Mapping `Partition 0 -> Source Subtask 0`, `Partition 1 -> Source Subtask 1` không nên hiểu là cố định mãi mãi. Flink Kafka Source sẽ phân phối các Kafka partitions cho các source subtasks dựa trên số partition và parallelism tại thời điểm chạy.

Nếu có 10 Kafka partitions nhưng source parallelism = 2, thì 2 source subtasks sẽ chia nhau đọc 10 partitions, ví dụ mỗi subtask đọc khoảng 5 partitions. Throughput nguồn vẫn bị giới hạn bởi 2 subtasks xử lý song song, dù Kafka có nhiều partition hơn.

Câu 3 — Source Parallelism lớn hơn số Kafka Partitions

? Câu hỏi

Ngược lại, nếu Kafka chỉ có 2 partitions nhưng em đặt Kafka Source parallelism = 4, có phải cả 4 source subtasks đều xử lý song song và throughput tăng gấp đôi không?

✓ Trả lời

Không. Nếu Kafka chỉ có 2 partitions nhưng source parallelism = 4, thì tối đa chỉ có 2 source subtasks thật sự đọc dữ liệu từ Kafka partitions. Hai subtask còn lại có thể idle vì không có partition để đọc.

Vì vậy throughput đọc từ Kafka không tự động tăng gấp đôi chỉ vì tăng source parallelism lên 4. Muốn tăng song song ở source thì thường cần tăng số Kafka partitions tương ứng.

Câu 4 — Partition, Subtask và Task Slot

? Câu hỏi

Demo của em có 2 task slots và parallelism 2.

Vậy có phải mỗi Kafka partition cần một Task Slot không? Quan hệ giữa partition, subtask và task slot thực sự là gì?

✓ Trả lời

Không phải mỗi Kafka partition cần một Task Slot.

Quan hệ đúng là:

- Kafka partition là đơn vị chia dữ liệu ở Kafka.
- Source subtask là instance song song của operator đọc dữ liệu.
- Task slot là tài nguyên thực thi trên TaskManager để chạy các subtasks.

Một source subtask có thể đọc nhiều Kafka partitions. Một task slot có thể chạy một chuỗi subtasks của nhiều operator nếu chúng được chain lại với nhau. Task slot là tài nguyên tính toán, còn partition là cách Kafka chia dữ liệu.

Câu 5 — Parallel Processing và Distributed Processing

? Câu hỏi

Em chỉ chạy 1 TaskManager nhưng parallelism = 2. Vậy em có thể gọi đây là parallel processing không? Và nó có phải distributed processing trên nhiều máy không?

✓ Trả lời

Có thể gọi là parallel processing, vì trong cùng một TaskManager vẫn có thể có 2 subtasks chạy song song bằng nhiều thread hoặc nhiều slot.

Nhưng đây chưa phải distributed processing trên nhiều máy. Distributed processing đúng nghĩa là job được chạy trên nhiều TaskManagers nằm trên nhiều máy khác nhau. Demo 1 TaskManager chỉ chứng minh song song cục bộ, chưa chứng minh phân tán trên cluster nhiều máy.

Câu 6 — HASH trong Job Graph và keyBy

? Câu hỏi

Trong Job Graph của em có đoạn:

Source → Parse/Filter → HASH → Analytics → Sink

Chữ HASH ở đây xuất hiện vì cái gì? Nếu bỏ keyBy(account_id) thì dataflow thay đổi như thế nào?

✓ Trả lời

Chữ 'HASH' xuất hiện vì có thao tác 'keyBy(account_id)'. Flink phải hash key 'account_id' để quyết định record đó được gửi sang downstream subtask nào.

Nếu bỏ 'keyBy(account_id)', dataflow sẽ không còn bước shuffle theo hash key. Dữ liệu sẽ đi tiếp theo kiểu phân phối bình thường giữa các operator, và Flink không còn đảm bảo cùng một 'account_id' luôn về cùng một subtask để tính state/window riêng cho account đó.

Câu 7 — Tại sao phải keyBy trước Window

? Câu hỏi

Tại sao phải keyBy(account_id) trước khi tính window? Nếu không keyBy mà aggregate luôn thì kết quả khác gì?

✓ Trả lời

Phải `keyBy(account\_id)` trước khi tính window vì yêu cầu là tính thống kê theo từng tài khoản. Khi keyBy, Flink tạo keyed stream, nghĩa là state và window được tách riêng theo từng `account\_id`.

Nếu không keyBy mà aggregate luôn, kết quả thường là aggregate toàn bộ stream hoặc aggregate theo cách không phân biệt account. Khi đó `acc-01`, `acc-02`, `acc-03` có thể bị cộng chung, không còn đúng với bài toán analytics theo từng tài khoản.

Câu 8 — Một triệu Key có cần một triệu Subtask?

? Câu hỏi

Em nói keyBy đảm bảo cùng account_id đi về cùng downstream subtask. Nếu có 1 triệu account nhưng chỉ 2 subtasks thì Flink xử lý như thế nào? Có phải cần 1 triệu subtasks không?

✓ Trả lời

Không cần 1 triệu subtasks. Flink không tạo một subtask cho mỗi account.

Flink hash 1 triệu keys đó vào số downstream subtasks hiện có. Nếu chỉ có 2 subtasks, mỗi subtask sẽ quản lý một phần key-group/key range, có thể hiểu là mỗi subtask xử lý khoảng một nửa tập account. State vẫn được lưu riêng theo từng key, nhưng các key được gom vào số lượng subtasks nhỏ hơn nhiều.

Câu 9 — Hot Key và Data Skew

? Câu hỏi

Nếu acc-01 có cực kỳ nhiều transaction trong khi các account khác rất ít thì chuyện gì xảy ra? keyBy có tự động chia transaction của acc-01 sang nhiều subtasks để cân bằng tải không?

Đây là câu mình rất thích hỏi vì nó kiểm tra xem sinh viên có hiểu hot key/data skew hay chỉ thuộc "keyBy = chia dữ liệu".

✓ Trả lời

Nếu `acc-01` có quá nhiều transaction, subtask nhận key `acc-01` sẽ bị tải nặng hơn các subtask khác. Đây là hiện tượng hot key hoặc data skew.

`keyBy(account\_id)` không tự động chia transaction của cùng một key sang nhiều subtasks, vì làm vậy sẽ phá vỡ đảm bảo rằng state/window của một key nằm cùng một nơi. Muốn xử lý hot key cần thiết kế thêm, ví dụ chia key phụ, pre-aggregate nhiều nhánh rồi merge lại, hoặc dùng chiến lược riêng cho account quá nóng.

Câu 10 — Tại sao dùng Tumbling Window 10 giây?

? Câu hỏi

Tại sao em phải dùng 10-second Tumbling Window? Tại sao không tính tổng transaction của acc-01 từ lúc job bắt đầu chạy đến mãi về sau?

✓ Trả lời

Tumbling window 10 giây giúp giới hạn phạm vi tính toán theo từng khoảng thời gian ngắn, phù hợp với real-time analytics: mỗi 10 giây có một kết quả mới, state không tăng vô hạn, và hệ thống dễ quan sát.

Nếu tính tổng từ lúc job bắt đầu đến mãi về sau, đó là dạng unbounded aggregation. State của mỗi account sẽ tồn tại và tăng liên tục, kết quả ít phản ánh tình hình hiện tại, và hệ thống khó kiểm soát bộ nhớ/state hơn.

Câu 11 — Tumbling Window và Sliding Window

? Câu hỏi

Tumbling window khác Sliding window như thế nào? Nếu đổi demo này sang sliding window 10 seconds / slide 5 seconds thì một transaction có thể thuộc bao nhiêu window?

✓ Trả lời

Tumbling window là các cửa sổ không chồng lấn, ví dụ 10:00:00-10:00:10 rồi 10:00:10-10:00:20. Mỗi event chỉ thuộc một window.

Sliding window có thể chồng lấn. Nếu dùng window size 10 giây và slide 5 giây, các window bắt đầu mỗi 5 giây nhưng kéo dài 10 giây. Một transaction thường có thể thuộc 2 windows.

Câu 12 — Window State được lưu ở đâu?

? Câu hỏi

Trong 10 giây đó, Flink nhận:

acc-01 \$100

acc-01 \$300

acc-01 \$500

nhưng window chưa kết thúc. Vậy count=3 và total=900 được giữ ở đâu? Đây có phải state không?

✓ Trả lời

Trong lúc window chưa kết thúc, các giá trị như `count=3` và `total=900` được giữ trong state của Flink, cụ thể là keyed window state nếu stream đã `keyBy(account\_id)`.

Đúng, đây là state. Khi window đóng hoặc được trigger, Flink lấy state đó để phát ra kết quả aggregate, sau đó có thể cleanup state của window theo cơ chế window cleanup.

Câu 13 — Processing Time và Event Time

? Câu hỏi

Window 10 giây của em đang dựa trên Processing Time hay Event Time? Nếu transaction xảy ra lúc 10:00:05 nhưng đến Flink lúc 10:00:20 thì nó thuộc window nào?

Câu này có khả năng bị hỏi cao vì event của demo có trường `event_time`. Report cho thấy transaction có `event_time`, nhưng phần report hiện tại không đủ để mình kết luận job thực sự assign timestamp/watermark thế nào.

✓ Trả lời

Phải xem code demo đang dùng Processing Time hay Event Time.

Nếu dùng Processing Time, transaction đến Flink lúc 10:00:20 thì nó thuộc window theo thời điểm xử lý 10:00:20, dù `event_time` trong dữ liệu là 10:00:05.

Nếu dùng Event Time và có assign timestamp/watermark từ trường `event_time`, transaction đó thuộc window chứa mốc 10:00:05. Tuy nhiên nếu nó đến quá trễ so với watermark và allowed lateness, nó có thể bị xem là late event và bị drop hoặc xử lý theo side output tùy cấu hình.

Câu 14 — Vai trò của Watermark

? Câu hỏi

Nếu em dùng Event Time thì Watermark có vai trò gì? Watermark được tạo ở đâu và nó ảnh hưởng việc đóng/trigger window thế nào?

✓ Trả lời

Watermark dùng trong Event Time để báo cho Flink biết tiến độ thời gian sự kiện đã đi đến đâu. Nói đơn giản, watermark là tín hiệu rằng hệ thống không kỳ vọng sẽ nhận thêm event cũ hơn mốc watermark, ngoại trừ phần lateness được cho phép.

Watermark thường được tạo ở source hoặc ngay sau source bằng `WatermarkStrategy`, dựa trên trường timestamp của event. Window event-time chỉ được trigger/đóng khi watermark vượt qua end time của window. Vì vậy watermark quyết định khi nào Flink có thể xuất kết quả cho một window dựa trên event time.

Câu 15 — Checkpoint thực sự lưu gì?

? Câu hỏi

Em show cho thầy:

Checkpoint

COMPLETED

Vậy checkpoint thực sự lưu cái gì? Nó có lưu toàn bộ transaction Kafka vào checkpoint không?

✓ Trả lời

Checkpoint lưu trạng thái cần thiết để khôi phục job nhất quán, ví dụ:

- Source progress, như Kafka offsets đã đọc đến đâu.
- Operator state.
- Keyed state.
- Window state đang tích lũy.
- Metadata cần cho consistent recovery.

Checkpoint không lưu toàn bộ transaction Kafka. Dữ liệu gốc vẫn nằm trong Kafka theo retention. Flink chỉ lưu offset và state xử lý để khi restart có thể đọc lại đúng đoạn cần đọc và khôi phục state đúng thời điểm checkpoint.

Câu 16 — Recovery khi TaskManager bị lỗi

? Câu hỏi

Em cấu hình checkpoint mỗi 10 giây.

Nếu TaskManager chết ở giây thứ 17 thì Flink khôi phục từ đâu? Những transaction từ checkpoint gần nhất đến lúc crash có bị mất không?

✓ Trả lời

Nếu checkpoint mỗi 10 giây và TaskManager chết ở giây 17, Flink sẽ khôi phục từ checkpoint hoàn thành gần nhất, ví dụ checkpoint ở giây 10 nếu checkpoint đó đã `COMPLETED`.

Các transaction từ sau checkpoint gần nhất đến lúc crash không bị mất nếu Kafka vẫn giữ dữ liệu. Flink sẽ dùng Kafka offset trong checkpoint để đọc lại từ vị trí đã lưu. Một số record có thể được xử lý lại, nhưng với checkpoint và sink hỗ trợ đúng cơ chế, kết quả cuối có thể vẫn exactly-once.

Câu 17 — Kafka đã lưu dữ liệu, tại sao vẫn cần Checkpoint?

? Câu hỏi

Kafka đã lưu transaction rồi, vậy tại sao Flink vẫn cần checkpoint? Kafka có dữ liệu thì restart rồi đọc lại chẳng phải được sao?

Đây là câu phản biện rất đáng chuẩn bị. Câu trả lời phải liên hệ được:

Kafka

→ input records / offsets

Flink checkpoint

- operator state
- window state
- source progress
- consistent recovery

chứ không chỉ nói "checkpoint để backup".

✓ Trả lời

Kafka lưu input records, nhưng Kafka không biết state nội bộ của Flink.

Flink vẫn cần checkpoint vì ngoài Kafka offsets, job còn có window state, keyed state, operator state và tiến độ xử lý nhất quán giữa các operator. Nếu chỉ đọc lại Kafka mà không khôi phục state đúng thời điểm, kết quả window/aggregate có thể sai, bị cộng thiếu hoặc cộng trùng.

Vì vậy Kafka giúp replay input, còn Flink checkpoint giúp khôi phục toàn bộ trạng thái xử lý một cách consistent.

Câu 18 — Checkpoint có đồng nghĩa Exactly-once?

? Câu hỏi

Trong slide em nói Flink hỗ trợ Exactly-once. Vậy chỉ cần enableCheckpointing() là toàn bộ hệ thống Kafka → Flink → Sink của em chắc chắn exactly-once chưa?

✓ Trả lời

Chưa chắc. `enableCheckpointing()` là điều kiện quan trọng, nhưng chưa đủ để toàn bộ pipeline Kafka → Flink → Sink chắc chắn exactly-once.

Exactly-once phụ thuộc vào cả source, Flink state và sink. Kafka source cần lưu offset trong checkpoint. Flink state phải checkpoint được. Sink cũng phải hỗ trợ exactly-once, ví dụ transactional sink, two-phase commit, hoặc ghi idempotent/upsert theo khóa. Nếu sink chỉ ghi thường và không chống duplicate, hệ thống có thể vẫn bị duplicate khi recover.

Câu 19 — Console Sink, Duplicate và Database Sink

? Câu hỏi

Demo hiện tại Sink chỉ print ra console.

Nếu job crash, restore checkpoint và xử lý lại một số transaction, console có thể in duplicate không? Nếu thay bằng database thì em giải quyết chuyện duplicate như thế nào?

✓ Trả lời

Có. Console sink có thể in duplicate nếu job crash rồi restore checkpoint và xử lý lại một số transaction. Console không phải transactional sink và không thể rollback những dòng đã in ra trước crash.

Nếu thay bằng database, có thể xử lý duplicate bằng các cách như:

- Ghi idempotent theo khóa duy nhất, ví dụ `transaction\_id` hoặc `(account\_id, window\_start, window\_end)`.
- Dùng upsert thay vì insert mù.
- Dùng transaction/two-phase commit nếu connector và database hỗ trợ.
- Thiết kế bảng kết quả sao cho ghi lại cùng một window sẽ overwrite kết quả cũ thay vì tạo dòng trùng.

Câu 20 — Control Flow và Data Flow

? Câu hỏi

JobManager chết thì transaction có đi qua JobManager không? Hãy vẽ cho thầy control flow và data flow khác nhau ở đâu.

Mình sẽ cố tình chờ xem bạn có vẽ sai:

Kafka → JobManager → TaskManager ❌

hay hiểu đúng:

JobManager

|

control/coordination

↓

Kafka —————→ TaskManager —————→ Sink

↑

data flow



Và câu cuối mình sẽ dùng để phân loại mức hiểu



Trả lời

Transaction không đi qua JobManager. JobManager chủ yếu làm control flow: nhận job, lập lịch, điều phối checkpoint, quản lý metadata và recovery.

Data flow đi trực tiếp giữa Kafka, TaskManager và Sink:

```
```text
```

```
JobManager
```

```

|
control/coordination

v

Kafka -----> TaskManager -----> Sink

^

data flow

...

```

Vì vậy sơ đồ `Kafka -> JobManager -> TaskManager` là sai nếu nói về luồng dữ liệu transaction.

## Câu 21 — Scale demo khi có thêm máy

### ? Câu hỏi

"Nếu thầy cho em thêm 3 máy nữa, em sẽ thay đổi kiến trúc demo hiện tại như thế nào để tận dụng chúng? Em có cần sửa logic keyBy → window → aggregate không?"

Nếu trả lời tốt câu này thì chứng tỏ bạn hiểu một điểm rất quan trọng:

LOGIC

Kafka

↓

keyBy

↓

Window

↓

Aggregate

không nhất thiết thay đổi, trong khi physical deployment có thể scale từ:

Laptop

└─ 1 TaskManager

thành:

Cluster

└─ Machine A → TaskManager

└─ Machine B → TaskManager

└─ Machine C → TaskManager

└─ Machine D → TaskManager

rồi tăng partitions/parallelism/resources phù hợp.

### ✅ Trả lời

Nếu có thêm 3 máy, em sẽ scale phần physical deployment chứ không nhất thiết sửa logic xử lý.

Logic vẫn có thể giữ nguyên:

```
```text
```

```
Kafka
```

```
|
```

```
keyBy(account_id)
```

```
|
```

```
Window
```

```
|
```

```
Aggregate
```

```
```
```

Điều cần thay đổi là triển khai nhiều TaskManagers trên nhiều máy, tăng tổng task slots, tăng parallelism của job cho phù hợp, và nếu source đang là bottleneck thì tăng số Kafka partitions. Khi đó Flink có thể phân phối subtasks sang nhiều máy hơn.

Em không cần sửa logic `keyBy -> window -> aggregate` chỉ để tận dụng thêm máy. Nhưng em cần đảm bảo số Kafka partitions, parallelism, task slots và tài nguyên cluster được cấu hình hợp lý. Nếu có hot key như `acc-01` quá nặng, lúc đó mới cần cân nhắc sửa logic để xử lý data skew.

---